

BUDGETWISE

A Cloud-Native Web Platform for Personal Budget Management

Project Report Cloud-Native Web System

Application domain: Fintech / Personal Finance

June 2026

Table of Contents

1. Introduction and Problem Context	3
2. Literature Review and Problem Justification.....	3
3. System Architecture	4
4. Justification of Technology Choices	5
5. Implementation	6
6. Security.....	7
7. Scalability and Cloud Deployment.....	7
8. UX Design and Visual Identity.....	8
9. Testing and Validation.....	8
10. Conclusion and Future Work	9
References	9

1. Introduction and Problem Context

Managing personal finances remains one of the most common everyday challenges, regardless of income level. A significant share of households, particularly across French-speaking Sub-Saharan Africa, have no structured tool for tracking their spending, relying instead on memory, paper notebooks, or spreadsheets that are poorly suited to mobile, day-to-day use. This lack of real-time visibility into personal cash flow directly contributes to overspending, an inability to anticipate difficult end-of-month periods, and insufficient savings.

BudgetWise addresses this concrete problem by offering a web platform accessible from any device, allowing a user to categorize transactions, set monthly budgets (either global or per spending category), and receive automatic alerts before a limit is exceeded rather than after the fact. The goal is not to reinvent personal accounting, but to make tracking simple and immediate enough that it becomes a lasting habit rather than a chore abandoned after a few weeks.

This report documents the full design cycle of the project: justification of the problem through a targeted literature review, the technical architecture adopted, the technology choices and their rationale, the actual implementation of the system, its cloud deployment strategy, and a critical discussion of security, scalability, and future directions.

2. Literature Review and Problem Justification

Several studies in behavioral economics have shown that simply visualizing spending on a regular basis significantly changes an individual's financial behavior. The principle of "mental accounting," formalized by Richard Thaler, shows that individuals mentally segment their money into categories, and that a tool which makes this segmentation explicit and visible reinforces budgeting discipline rather than imposing it artificially.

Personal Finance Management (PFM) applications such as Mint, YNAB (You Need A Budget), and, more locally, West African mobile money solutions, have popularized three features considered central by fintech product literature: (1) automatic or semi-automatic categorization of transactions, (2) the definition of budget envelopes with threshold alerts, and (3) trend visualization over time rather than a single snapshot. BudgetWise adopts these three pillars while remaining deliberately simpler than general-purpose Western tools, so that it stays usable without an automated bank connection—a relevant choice in contexts where open banking or bank APIs are not consistently available.

On the technical side, the software engineering literature on cloud-native architectures (notably the work behind the Twelve-Factor App methodology) recommends strict separation of environment configuration, statelessness of application processes, and externalization of data persistence. These principles directly guided BudgetWise's architecture: no data is stored in the local memory of the application server, all sensitive configuration (secrets, connection string) is injected through environment variables, and the compute layer (Vercel Serverless Functions) is fully decoupled from the persistence layer (MongoDB Atlas), enabling native horizontal scalability without code rewrites.

Finally, regarding the choice of a document-oriented database over a relational one: a user's personal financial data (transactions, categories, budgets) has a relatively stable structure but evolves unpredictably in cardinality (a user might have ten or ten thousand transactions). MongoDB, with its flexible document model and native indexing for temporal aggregation queries, proves particularly well suited to the typical queries of a financial dashboard (sums by category, monthly trends), which are naturally expressible through MongoDB's aggregation pipeline without complex joins.

3. System Architecture

3.1 Overview

BudgetWise adopts a unified full-stack architecture built on Next.js (App Router), which merges the user interface layer (React, server- and client-side rendering) and the API layer (serverless routes) within a single project. This choice eliminates the configuration duplication normally required between a separate frontend and backend, simplifies deployment to a single Vercel project, and reduces network latency since API calls remain internal to the same deployment.

The logical layout of the system can be summarized as follows:

- **Client (browser)** React application (Next.js App Router), Tailwind CSS for the design system, Recharts for data visualization.
- **API layer** Next.js routes (app/api/**/route.ts), executed as serverless functions on Vercel's infrastructure.
- **Business logic layer** Shared TypeScript modules (lib/): JWT authentication, Zod validation, budget-overrun detection logic.
- **Persistence layer** MongoDB Atlas (managed cloud cluster), accessed via Mongoose (ODM) with connection caching adapted to the serverless environment.
- **Hosting / CDN** Vercel (Edge Network), providing static asset distribution and serverless function execution close to the end user.

3.2 Data Model

The system relies on five MongoDB collections, modeled through strict Mongoose schemas that enforce type consistency despite MongoDB's schema-less nature:

Collection	Role
User	Identity, hashed credentials, role (user/admin), preferred currency, indicative monthly income.
Category	Expense/income categories owned by each user, with color, icon, and an optional monthly limit.
Transaction	Individual financial movement: amount, type, associated category, date, description, tags.
Budget	Total spending limit defined per month/year for a user (unique compound key).

Notification	Automatically generated alerts (budget thresholds) or informational messages, with a read/unread status.
---------------------	--

Each Transaction and Category document references its owner (user field) as well as, for transactions, the associated category. Compound indexes (user + date, user + name + type) ensure the performance of filtering queries and the logical uniqueness of categories per user.

3.3 Authentication and Security Flow

Authentication relies on a JWT (JSON Web Token) scheme stored in an httpOnly cookie rather than in the browser's localStorage, in order to reduce the XSS attack surface: a malicious script injected client-side cannot read an httpOnly cookie. Passwords are never stored in plain text; they are hashed using bcrypt (automatic salting, cost factor 10) before any write to the database.

The authentication cycle follows these steps:

1. The user submits their credentials through the login or registration form.
2. The API route validates the data format (Zod schema) before any database access.
3. On registration, the password is hashed and a default set of categories is created for the new user.
4. A signed JWT (containing the user ID, email, and role) is issued and placed in an httpOnly cookie, secured in production (Secure flag) and subject to a SameSite=Lax policy.
5. Every request to a protected route extracts and verifies this token server-side before executing business logic; administration routes additionally verify that the associated role is admin.

3.4 Architecture Diagram (textual representation)

The diagram below summarizes the main data flows between the layers of the system:

```

[ Browser / Mobile ]
  □ HTTPS (Vercel Edge Network)
[ Next.js App Router  React Pages (SSR/CSR) ]
  □ fetch() / httpOnly cookies (JWT)
[ Next.js API Routes  Vercel Serverless Functions ]
  ↑↓ Zod validation → Business logic (lib/)
  □ Mongoose ODM (cached connection pool)
[ MongoDB Atlas  Managed Cloud Cluster (M0) ]
  
```

4. Justification of Technology Choices

4.1 Next.js (App Router) as a Full-Stack Framework

Next.js was chosen over a classic separate React + Express setup for three main reasons. First, it unifies deployment: a single Vercel project hosts both the interface and the API, simplifying CORS configuration (a non-issue since both share the same origin) and secrets management. Second, the file-based routing system (`app/api/**/route.ts`) makes the project structure immediately readable and consistent with REST conventions. Third, Next.js benefits from first-class integration with Vercel, its publisher, which greatly reduces deployment friction a decisive factor for this project, whose goal was a functioning deployment without manual server configuration.

4.2 MongoDB Atlas and Mongoose

MongoDB Atlas was selected for its permanently free tier (M0 cluster), its fully managed operations (backups, security updates, monitoring), and its native compatibility with serverless environments via the Node.js driver. Mongoose was added as an ODM (Object Document Mapper) layer to enforce strict schema validation at the application level a trade-off considered necessary for a financial system, where type consistency (positive numeric amounts, transaction-type enumerations, etc.) cannot be left to chance despite MongoDB's native flexibility.

A specific concern of serverless environments was addressed: each invocation of a Vercel function can potentially spin up a new execution instance, which risks multiplying database connections until the allowed pool is exhausted. The connection module (`lib/mongodb.ts`) therefore implements a process-level connection cache, ensuring that an existing connection is reused rather than recreated on every call.

4.3 JWT Authentication Rather Than Server-Side Sessions

A session system stored in a database (or in server memory) would have introduced shared state across serverless function invocations, contrary to the statelessness principle being pursued. Signed, self-contained JWTs allow each serverless function to independently validate a user's identity without access to shared state, which is ideally suited to an on-demand distributed architecture such as Vercel's.

4.4 Tailwind CSS and Recharts

Tailwind CSS was chosen for the speed of iteration it allows on a coherent design system (custom color palette, typography, spacing), without depending on a heavy third-party component library. Recharts was selected for data visualization (trend charts, category breakdowns) due to its natural declarative composition with React and its compatibility with Next.js's client-side rendering.

4.5 Data Validation with Zod

Every API route systematically validates incoming data through Zod schemas before any business processing or database access. This server-side validation is independent of any

client-side validation (which remains a UX convenience rather than a security guarantee), in line with the principle that no data coming from the client should be trusted by default.

5. Implementation

5.1 Delivered Features

The system implements the following set of fully functional features:

- Complete authentication: registration, login, logout, session persistence via httpOnly cookie.
- Role management: the first account created on the platform is automatically promoted to administrator; roles can subsequently be managed from the admin console.
- Full transaction CRUD, with filters (type, category, text search) and server-side pagination.
- Full category CRUD, with customization (color, icon) and an optional monthly limit per category.
- Monthly global budget definition, with a visual progress bar (green / amber / red depending on the threshold reached).
- Automatic notification system: detection of 80% and 100% thresholds (global and per-category budgets), with deduplication to avoid spamming repeated notifications.
- Analytical dashboard: key indicators (income, expenses, net balance), a 6-month trend chart, expense breakdown by category (donut chart).
- Administration console: platform-wide statistics (number of users, transactions, total financial volume, sign-up trend), user account management (promotion, demotion, deletion).
- Fully responsive interface, with a sidebar navigation on desktop and a bottom navigation bar on mobile.

5.2 Code Quality and Best Practices

The project is written entirely in strict TypeScript, with shared types between the client and server (src/types directory). The structure follows a clear separation of concerns: generic, reusable UI components (Button, Input, Modal, Card) are isolated from business logic, which is itself centralized in dedicated modules (lib/auth.ts, lib/budget-alerts.ts, lib/validation.ts). Every API response follows a standardized format (`{ success, data }` or `{ success: false, error }`), making client-side error handling predictable.

The code was validated by two automated tools prior to delivery: the TypeScript compiler (`tsc --noEmit, no errors`) and ESLint with Next.js's strict configuration (`eslint-config-next`, including the `react-hooks` rule set), also returning zero errors and zero warnings after cleanup.

5.3 Illustrative Business Logic: Budget Overrun Detection

One of the most representative pieces of business logic in the project is the budget-overrun detection service (lib/budget-alerts.ts), triggered after every creation or modification of an

expense-type transaction. This service runs two independent checks through the MongoDB aggregation pipeline: the first computes the sum of the current month's expenses and compares it to the user's global budget; the second performs the same calculation at the category level if a monthly limit has been associated with it. In both cases, an 80% threshold generates a warning notification, while a 100% threshold generates an overrun notification. A deduplication mechanism prevents identical notifications from being created within a six-hour window, avoiding redundant messages when transactions occur in quick succession.

6. Security

Security was addressed across several complementary layers:

- Password hashing via bcrypt (never stored in plain text), with automatic per-record salting.
- JWT tokens signed with a dedicated secret, stored in an httpOnly cookie, marked secure (HTTPS-only in production) and SameSite=Lax, limiting exposure to classic XSS and CSRF attacks.
- Strict per-user data isolation: every read/write request is systematically filtered by the authenticated user's ID extracted from the token, preventing any cross-account data access.
- Role-based access control (simplified RBAC) for administration routes, verified server-side on every request rather than client-side only.
- Systematic input validation via Zod, reducing the risk of injection or malformed data reaching the persistence layer.
- Database network access restriction, configurable through MongoDB Atlas's IP allow-list.
- Sensitive variables (connection string, JWT secret) exclusively managed through the deployment platform's environment variables, never committed to the source code.

A recommended future improvement would be adding rate limiting on authentication routes to reduce exposure to brute-force attacks, as well as introducing email verification at registration.

7. Scalability and Cloud Deployment

The choice of a serverless architecture (Vercel) paired with a managed database (MongoDB Atlas) gives the system native horizontal scalability: every HTTP request can be handled by an independent function instance, provisioned on demand by the platform, without manual load-balancer configuration. The MongoDB connection caching described in section 4.2 ensures that this elasticity does not degrade database performance as the number of concurrent instances grows.

Deployment relies on a simple but effective continuous integration pipeline: every push to the main branch of the GitHub repository automatically triggers a new build and deployment on Vercel, running the TypeScript compiler and Next.js in strict mode — any build that fails to compile is automatically blocked before reaching production, providing a native form of continuous quality guardrail (CI/CD).

For larger-scale growth, the natural evolution of the system would involve: moving from a free, shared MongoDB M0 cluster to a dedicated tier with multi-region replication; introducing a caching layer (Redis) for the most frequently requested dashboard aggregations; and offloading notification generation to an asynchronous queue rather than synchronous post-transaction processing, for very high-volume use cases.

8. UX Design and Visual Identity

The interface was designed around a distinct identity, conceived specifically for this product rather than conforming to the generic visual conventions of a SaaS dashboard. The chosen palette pairs a deep sage green symbolizing growth and savings with a restrained coral reserved for expenses and alerts, set against a warm paper background rather than a clinical white. The typography combines a display typeface (Fraunces, a contemporary serif) for headings with a sans-serif typeface (Inter) for body text, and a monospace typeface (JetBrains Mono) dedicated exclusively to displaying monetary amounts—a choice that vertically aligns digits and reinforces financial legibility.

The signature design element is a scrolling "stock-ticker" style banner, displayed at the top of the dashboard and landing page, continuously presenting the month's key financial indicators. This choice visually anchors the product's fintech identity from the very first interaction, without resorting to the generic visual templates most commonly seen in automatically generated interfaces.

The entire interface follows baseline accessibility standards (visible keyboard focus, ARIA labels on icon buttons, respect for prefers-reduced-motion to disable animations for motion-sensitive users) and remains fully responsive, with sidebar navigation reorganized into a bottom navigation bar on mobile screens.

9. Testing and Validation

System validation was carried out at several levels prior to delivery:

- Static type checking: the entire TypeScript codebase compiles without error (`tsc --noEmit`).
- Static quality analysis: ESLint (Next.js strict configuration, including the react-hooks rule set) reports zero errors and zero warnings.
- Production build: the project was successfully compiled via next build, generating the full expected set of static and dynamic routes (pages and serverless API functions).
- Manual functional validation of critical user journeys: registration, login, creating/editing/deleting transactions and categories, setting a budget, triggering a threshold notification, accessing and using the administrator console.

For a more comprehensive automated test suite, adding unit tests (Vitest or Jest) for critical business-logic modules (`lib/budget-alerts.ts`, `lib/auth.ts`), as well as end-to-end integration tests (Playwright) covering the authentication and transaction flows, would be the natural next step, not covered within the scope of this initial delivery.

10. Conclusion and Future Work

BudgetWise demonstrates the feasibility of a complete, secure, production-deployable fintech platform, built entirely on a modern cloud-native stack (Next.js, MongoDB Atlas, Vercel) with no server infrastructure to manage manually. The system directly addresses the identified problem the lack of accessible personal budget-tracking tools by combining a polished user experience, proactive rather than reactive alerts, and a technical architecture aligned with current software engineering best practices (separation of concerns, systematic validation, statelessness, security by default).

Natural avenues for future development include: integrating bank or mobile-money APIs for automatic transaction synchronization, adding personalized savings goals with projections, a native mobile application sharing the same API, and generic budgeting recommendations based on a user's spending history. The current architecture, modular and strictly typed, was designed to accommodate these evolutions without a major structural rewrite.

References

- Thaler, R. H. (1999). Mental Accounting Matters. *Journal of Behavioral Decision Making*, 12(3), 183–206.
- Wiggins, A. (2017). The Twelve-Factor App. <https://12factor.net>
- MongoDB Inc. (2026). MongoDB Atlas Documentation. <https://www.mongodb.com/docs/atlas/>
- Vercel Inc. (2026). Vercel Documentation Next.js Deployment. <https://vercel.com/docs>
- Next.js Documentation (2026). App Router. <https://nextjs.org/docs/app>
- OWASP Foundation (2025). OWASP Top 10 Web Application Security Risks. <https://owasp.org/Top10/>